

pPOP: Fast yet accurate parallel hierarchical clustering using partitioning

Manoranjan Dash ^{a,*}, Simona Petrutiu ^b, Peter Scheuermann ^b

^a School of Computer Engineering, Nanyang Technological University, Blk N4, #2c-85, Nanyang Avenue, Singapore 639798, Singapore

^b Department of Electrical and Computer Engineering, Northwestern University, Evanston, IL 60208, United States

Received 11 January 2006; received in revised form 5 July 2006; accepted 6 July 2006

Available online 8 August 2006

Abstract

Hierarchical agglomerative clustering (HAC) is very useful but due to high CPU time and memory complexity its practical use is limited. Earlier, we proposed an efficient partitioning – partially overlapping partitioning (POP) – based on the fact that in HAC small and closely placed clusters are agglomerated initially, and only towards the end larger and distant clusters are agglomerated. Here, we present the parallel version of POP, pPOP. Theoretical analysis shows that, compared to the existing algorithms, pPOP achieves CPU time speed-up and memory scale-down of $O(c)$ without compromising accuracy where c is the number of cells in the partition. A shared memory implementation shows that pPOP outperforms existing algorithms significantly.

© 2006 Elsevier B.V. All rights reserved.

Keywords: Hierarchical agglomerative clustering; Partitioning; Parallel algorithm; Shared memory multiprocessor architecture

1. Introduction

Clustering or grouping of similar objects is one of the most widely used techniques in data mining. It is an important data exploration task used in diversified areas including market segmentation in business, gene categorization in biology, spatial discovery, and document classification on the WWW. Some surveys and books on clustering are found in [14,16,17]. A popular method of clustering is hierarchical agglomerative clustering (HAC). It starts with each point in separate clusters and iteratively agglomerates the closest pair of clusters in each iteration until all points belong to a single cluster. The final hierarchical cluster structure is called a *dendrogram* (see Fig. 1), a tree like structure that shows which clusters are agglomerated at each level. To obtain the best level and its corresponding clusters, each level of the dendrogram can be evaluated by a cluster validation method [21].

HAC algorithms are non-parametric,¹ natural and simple in grouping objects, and capable of finding clusters of different shapes by using different similarity measures. However, they have several drawbacks. First,

* Corresponding author. Tel.: +65 6790 6167; fax: +65 6792 6559.

E-mail address: asmdash@ntu.edu.sg (M. Dash).

¹ It does not assume any statistical distribution.

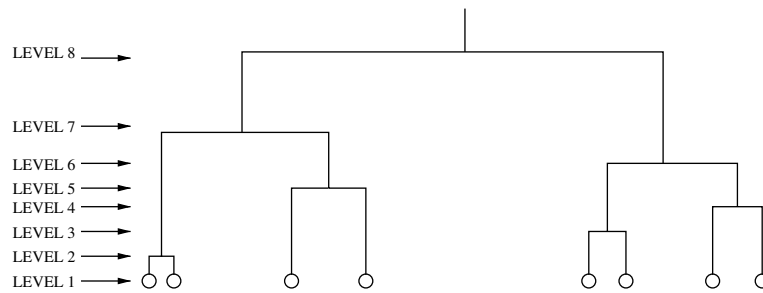


Fig. 1. Dendrogram: output of hierarchical clustering.

HAC algorithms have high CPU time and memory complexities. For example, an efficient algorithm that uses a stored matrix representation based on priority queues and a geometric metric for distances based on the centroid method has time complexity $O(N^2 \log N)$ [10]. We observe here that a number of techniques were proposed to expand HAC to large datasets, such as BIRCH [30] and CURE [13]. However, *they do not make the HAC algorithm itself faster*. Instead, they use approximations (e.g., summarized (BIRCH) or sampled (CURE) points) so that HAC can operate on a smaller dataset.

In [6,7,9] we have shown that the complexities of the existing sequential HAC algorithms can be reduced significantly by an efficient partitioning scheme without compromising accuracy. The proposed methods are based on the observation that in HAC most iterations agglomerate very small clusters separated by a very small dissimilarity. Only a small number of iterations towards the end agglomerate the large clusters. Using this observation, a structure called *partially overlapping partitioning* (POP) divides the data into a number of overlapping cells. Analyses and experiments showed that POP-based sequential HAC algorithms reduce the existing time and memory complexities by a factor close to the number of cells c – appropriately defined later.

To cluster increasingly massive datasets that are common in today's applications, a number of parallel algorithms have been proposed. For example see [11,19,22–24,29]. These algorithms either parallelize other types of clustering methods such as K -means [11], or perform subspace clustering [22]. The parallelizations of hierarchical clustering are limited to the divisive algorithm reported in [24] or to HAC algorithms that are not very efficient due to the lack of performance enhancing partitioning [19,20,23,29].

In this paper we present parallel versions of POP, called pPOP that are geared towards a shared memory multiprocessor architecture.² We present the pPOP version in detail for stored matrix using priority queue and compare it with the traditional parallel implementation. Since our implementation is done in OpenMP we give the details about scheduling, data dependencies and race conditions, as appropriate. We then proceed to show theoretically that pPOP, just like its sequential counterpart POP, reduces the time and memory complexities of the existing parallel algorithms by a factor close to the number of cells c for stored matrix versions of the algorithms. Next, we give results of our experimental evaluation over synthetic and benchmark datasets of pPOP versions for both stored matrix and stored data. The results show that for large datasets the pPOP algorithms obtain near linear speedup. In addition, we show that for stored matrix implementations, pPOP results in multiple orders of magnitude improvement in computation time and memory requirements over the existing parallel HAC algorithms.

In summary, parallel HAC algorithms can be improved by using an efficient partitioning scheme without compromising their accuracy. Although POP partitioning is independent of the parallel architecture (be it shared memory or distributed memory as in MPI – Message Passing Interface), we foresee that distributed memory architecture with its numerous communication overhead will not be as advantageous as the shared memory architecture. Due to fine grain parallelism the shared memory implementation should perform better than the distributed memory.

The paper is organized as follows. The next section gives background information that includes a discussion about an important property of HAC, a brief description of existing HAC algorithms, and POP partitioning. In Section 3 we introduce parallel versions of POP and give theoretical results for time and memory complexities

² A shorter version of this paper appeared in EuroPar 2004 proceedings [8].

while comparing them to existing parallel algorithms. Section 4 gives experimental results on a shared memory parallel architecture. Section 5 discusses high-dimensional data. The paper concludes in Section 6 with discussions on future work.

2. Background

Let us assume that our dataset consists of N data points or objects each with M dimensions or attributes in K clusters. We shall denote X_{ij} as the j th dimension value of i th data point where $i = 1 \dots N$ and $j = 1 \dots M$. We assume only real type data. Dissimilarity between two data points containing real data is measured by their Euclidean (L_2) distance. Other distance measures such as *Manhattan* distance can be used.

In the following we give an outline of the HAC algorithm up to the point of creating the complete dendrogram. After the dendrogram is created, a user should cut the dendrogram at some level depending on his requirement.

Algorithm: Hierarchical Agglomerative Clustering

Input: Data (N, M)

Output: Dendrogram

1. for $i = 1$ to $N - 1$
2. find the closest pair of clusters
3. merge the closest pair of clusters
4. return dendrogram

In step 2, depending on the similarity metric used, the closest pair identified in each iteration can vary. In the following section we discuss an important observation of HAC.

2.1. The 90–10 rule

We ran the centroid type HAC method over a 2-D dataset with 100 clusters and some noise (see Fig. 2(a)). In the centroid method, each cluster is represented by its centroid and the cluster pair having the closest centroids is merged in each iteration. Based on the HAC results we have plotted in Fig. 2(b)–(d) the closest pair distance (*distance plot*), the size of the larger cluster (*size plot*), and the total time up to each iteration (*time plot*) respectively. The X -axes of these plots stand for percentage of iterations of HAC.

Notice that most agglomerations except for a small portion towards the end have very small closest pair distance compared to the maximum closest pair distance. Similarly, the size of the clusters is also very small except for the last few iterations. In the time plot, initial agglomerations are more time consuming than later ones because the remaining number of clusters to be merged decreases as the algorithm progresses. We experimented with many datasets having varying characteristics. The characteristics chosen are those that affect the clustering of data. The datasets selected had noise and consisted of overlapping clusters with different shapes (e.g., spherical, arbitrary), and multiple resolutions. For varying M , N (typically large – at least a few thousand points), and K , the general trend is as follows: if the majority of points is inside clusters, the shape of the distance plot (and the size plot) is similar to the one shown in Fig. 2. A counter example is uniformly distributed high-dimensional data. But typical clustering applications assume that *data are not uniformly distributed but consist of some clusters* and therefore follow the trend.

We name these observations the *90–10 rule* and state it as follows:

in a dendrogram, most levels from the bottom merge pairs of very small clusters separated by a very small distance representing a small proportion of the maximum closest pair distance.

Note that the ‘90’ and ‘10’ are only symbolic and do *not* suggest that exactly 90% of the agglomerations merge clusters separated by a distance less than exactly 10% of the maximum closest pair distance. In fact, one can name it 80–20 or 70–30 but the idea is that until the last portion of agglomerations, clusters are very

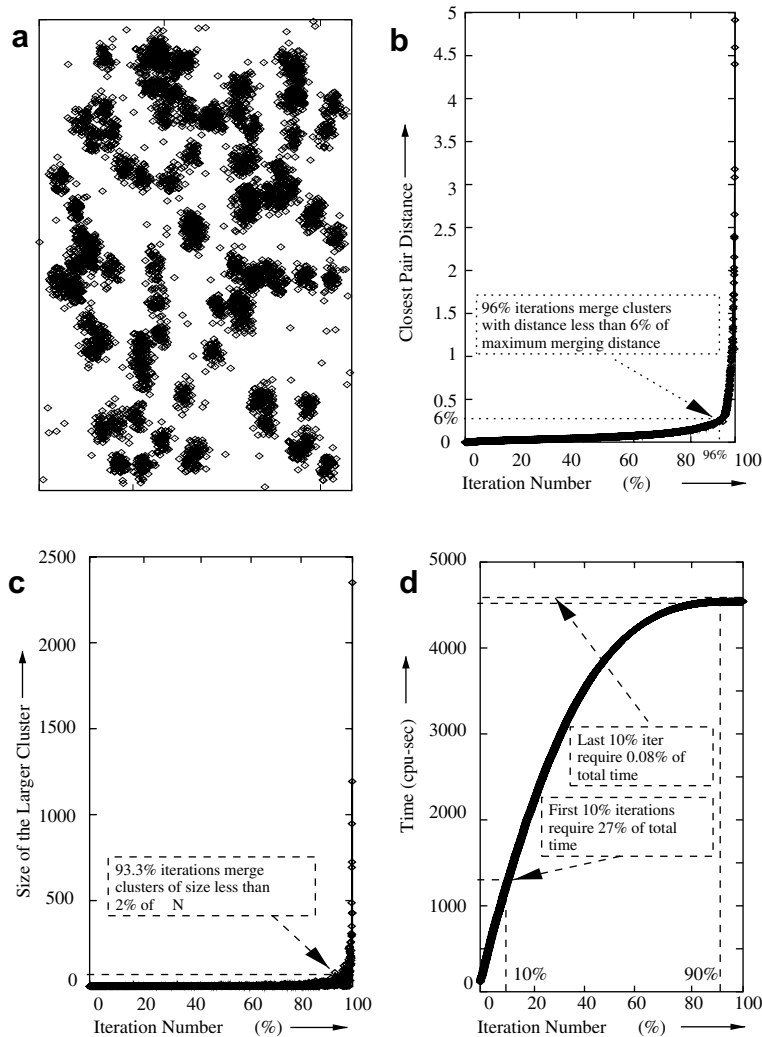


Fig. 2. The 90–10 rule: important observations about HAC: (a) dataset with 100 clusters, (b) distance plot, (c) size plot and (d) time plot.

close in proximity and very small in size. In place of the centroid method, one can use some other similarity method such as geometric type (e.g., centroid, median, minimum variance Ward's methods [27]) or graph type (e.g., single/complete/average link method). Application of the 90–10 rule to these methods is similar to its application to the centroid method. In the rest of the paper we limit our discussion to the centroid method and point the interested readers to [6,7] for details about how to apply the 90–10 rule to other similarity metrics.

2.2. Partially overlapping partitioning (POP)

Partially overlapping partitioning (POP) exploits the 90–10 rule to significantly reduce the time and memory requirements of HAC algorithms. The data is partitioned into c overlapping cells. The region of overlap between cells is called δ -region where δ is the overlapping distance. Fig. 3(a) shows an axis-parallel POP that divides the data-space uniformly into c cells ($c = 16$). For the centroid metric (and other geometric metrics) each cluster is represented by a single representative point. If the representative point of a cluster falls in a δ -region then each affected cell holds it, otherwise only one cell holds it. In Fig. 3(b) we show the same

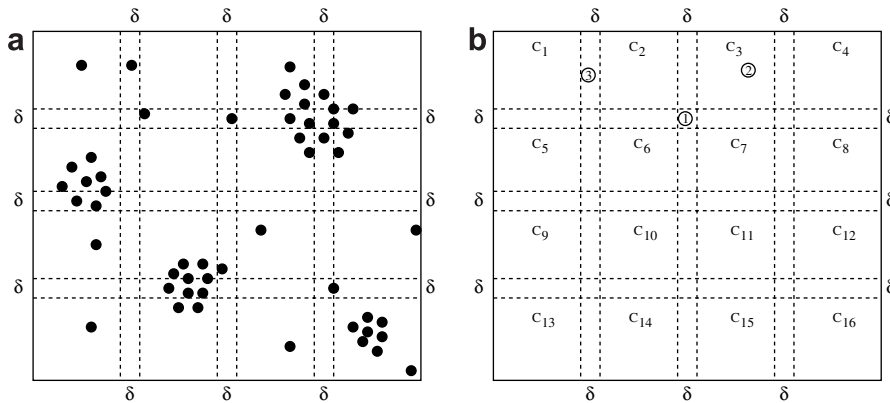


Fig. 3. Axis-parallel partially overlapping partitioning (POP).

partitioning, but this time the cells and clusters are numbered. Three clusters are selected from Fig. 3(a) in Fig. 3(b) to describe the POP partitioning. In Fig. 3(b) it can be seen that cluster 2 belongs to cell 3, cluster 3 belongs to cells 1 and 2, while cluster 1 belongs to cells 2, 3, 6 and 7.

We proposed a new 2-phase algorithm for HAC based on POP. In *phase 1* the data is partitioned into many overlapping cells. The basic idea is that, in each iteration the closest pair of clusters is found for each cell, and from those pairs the overall closest pair is found. If the overall closest pair distance is less than δ then the pair is merged. Distances between clusters of ONLY the container cell are affected and all other cells remain unaffected. If the closest pair or the newly merged cluster is in a δ -region then, in addition to the container cell, only the distances between clusters of the affected cells are updated. *Phase 1* terminates when the closest pair distance exceeds δ . *Phase 2* merges the remaining clusters of *phase 1* using a traditional algorithm, thus completing the dendrogram. Note that, by setting δ to the closest pair distance at the turning point of distance plot of Fig. 2, a large number of small clusters are merged in *phase 1* while only a small number of larger clusters are merged in *phase 2*. Assuming a small δ -region, the remaining number of clusters after *phase 1* is very small, i.e., *phase 2* time is negligible. In this way substantial reductions in the memory and CPU time are achieved. Empirical study shows a reduction factor of $O(c)$ for both memory and time complexities. This is possible because merging of clusters and updating of distances are required for only one cell most of the time. As shown in Fig. 2, the 2-phase algorithm performs best when δ is ideally set to the distance corresponding to the turning point of the distance plot. But in the absence of any information about the turning point before performing HAC, we devised a nested algorithm that starts with a small δ and a large c , and gradually increases δ and decreases c . The user can vary the amount of increase (δ_{incr}) and decrease (c_{decr}) in each step. Empirical study suggests that in each iteration $\delta_{\text{incr}} = c_{\text{decr}} = 10\%$ gives good performance. Initially δ is set to a very small value; if the user finds it difficult to set this initial value of δ then a possible initial value is 0. For the next nested iteration, δ is set to $(1 + \delta_{\text{incr}})$ times the last closest pair distance which exceeded the previous δ . Based on empirical study, the initial value of c ideally should be such that the average number of clusters per cell is between 5 and 20.

2.2.1. Accuracy

It is noteworthy that *phase 1* of POP ensures that *any pair with distance less than δ MUST reside together in at least one cell*. This is because of the way POP partitioning is performed. This ensures that *phase 1* will be continued while there exists at least one pair of clusters with distance less than δ . *Phase 1* terminates when the overall closest pair distance becomes equal to or greater than δ . It means that up to the termination of *phase 1*, the dendrogram generation is accurate. *Phase 2* is the traditional HAC itself over the remaining clusters. So, the 2-phase algorithm guarantees a correct dendrogram just like the traditional HAC. Note that ties between the closest pair distances can produce different dendrograms depending on which pair is chosen to be merged. There is no unique way of resolving these ties.

3. pPOP algorithms

In this section we describe the parallel version of POP, pPOP, and compare it with some existing methods. First of all we give a brief overview of some important sequential HAC algorithms which are found in the literature for parallel HAC algorithms. Then we give details of pPOP.

3.1. Complexities of existing sequential HAC algorithms

The first centroid algorithm proposed uses a dissimilarity matrix that stores dissimilarities between clusters. Its time complexity is $O(N^3)$ as the dissimilarity matrix of size $O(N^2)$ is searched $N - 1$ times to complete the dendrogram. The memory complexity is $O(N^2)$ because of the dissimilarity matrix. Two new data structures were suggested in [10] to improve the time complexity of this algorithm. The first method maintains the nearest neighbor of each cluster and their distances. In each iteration, the smallest nearest neighbor distance is found and the corresponding clusters are merged and the nearest neighbors of the affected clusters are updated. This method reduces the time complexity to $O(\alpha N^2)$ where α is the average number of clusters affected by each agglomeration. The memory complexity is reduced to $O(N)$. The second method maintains a heap-based priority queue for each cluster. In each iteration the top elements of all priority queues are compared, the smallest distance is selected, and the corresponding clusters are merged. Finally all the priority queues are updated. Because the priority queues have the property that insert and delete operations require $O(\log n)$ time for n items in the queue, the time complexity is $O(N^2 \log N)$ and the memory complexity is $O(N^2)$. In [1] the dissimilarity matrix and the priority queue algorithms are called *stored matrix* type whereas the nearest neighbor algorithm is labelled as *stored data* type.

3.2. Parallel HAC: related work and limitations

Some of the important results in parallel HAC algorithms have been reported in [19,20,23,29]. In [19,20] Li and Fang proposed parallel HAC algorithms on SIMD (Single Instruction Multiple Data) machines. This parallel architecture uses a parallel memory system and an alignment network to facilitate parallel access of both similarity matrix and data. For a dataset with N points, the number of memory accesses is reduced from $O(N^3)$ on a sequential machine to $O\left(\frac{N^3}{p}\right)$ on an SIMD machine with p processors. Thus, they reduced the time complexity by a factor of p for both single-link and complete-link HAC. The total memory requirements of $O(N^2)$ is not reduced. In [29] parallel algorithms for single-link HAC were proposed for arrays with reconfigurable optical buses. Olson [23] gave optimal algorithms for HAC using PRAM, butterfly, and tree architectures. The common feature of the above algorithms is: for stored matrix type the task of computing and maintaining $O(N^2)$ dissimilarities is divided among processors, whereas for stored data type the task of computing and maintaining the $O(N)$ nearest neighbors is divided among processors. For example, using p processors Olson showed that the time complexity of the dissimilarity matrix method reduces to $O\left(\frac{N^3}{p}\right)$ and that of the priority queue method reduces to $O\left(\frac{N^2 \log N}{p}\right)$. The time complexity for the nearest neighbor array method reduces to $O\left(\frac{\alpha N^2}{p}\right)$.

These algorithms are not very efficient because they still require $O(N^2)$ total memory for stored matrix type, and in each iteration they require to update all priority queues or the entire dissimilarity matrix. For stored data type, the algorithms check all the clusters after each agglomeration to determine whether the newly merged cluster is nearer than the previous nearest cluster. So, *the time reduction in these parallel algorithms is mainly due to parallelization, but not due to efficient partitioning.*

In this section and the next, we show both analytically and experimentally that efficient partitioning of the data by pPOP significantly improves the performance of parallel HAC algorithms. It outperforms the existing algorithms both in time and memory requirements. Parallelization using pPOP is done by performing all the tasks related to a *cell* in one processor, and dividing all the cells among different processors. Note that this strategy is different from the existing strategy where instead of a cell, tasks related to each cluster are performed in one processor, and the clusters themselves are divided among different processors. Thus, from

the beginning, pPOP has the advantage that instead of the usual $O(N^2)$ size, it only needs to create or maintain priority queues or a dissimilarity matrix of the order of $O\left(\frac{N^2}{c}\right)$ (assuming negligible δ region), where c is the number of cells. This is illustrated in Figs. 4 and 5, where the priority queues of the traditional parallel approach, denoted by pTRAD, and those of pPOP are illustrated respectively. In the pTRAD method, for each cluster we need to keep a priority queue to hold the distance between itself and every other cluster in the space. On the other hand, using pPOP, the priority queues are restricted to a cell only: for each cluster its priority queue holds the distances between itself and the clusters in the same cell only. This is shown pictorially in Fig. 5 where 12 data points are divided among 3 cells and for each cell the priority queues are restricted to the clusters in that cell only. But Fig. 4 shows that in the traditional approach each priority must hold the distances between itself and all other clusters.

As we will discuss in more detail shortly, the advantage of maintaining the priority queues on a cell basis is the processing gain obtained in the “merge and update step” when, for most of the time, it suffices to update the priority queues or similarity matrices of the clusters in the container cell only. We proceed now with the two algorithms for stored matrix type using priority queues.

3.3. pTRAD: existing parallel stored matrix algorithm using priority queues

Table 1 shows the existing parallel stored matrix algorithm using priority queues. The initial step of creating the priority queues is divided among the processors using simple scheduling. Thus, each processor computes the priority queues for $\frac{N}{p}$ clusters. In step 5, each processor searches the top elements of its own $\frac{N}{p}$ priority queues, and gets its own smallest distance. Since the amount of work on each processor is about the same, this kind of scheduling requires the least amount of communication. After this step, each processor will have the smallest distance it encountered (i.e., there will be p smallest distances, one for each processor). From these p distances the overall smallest distance is computed (steps 6–7). In this step there is a need for synchronization because only one processor can compare its distance with the overall smallest distance at any time; if more

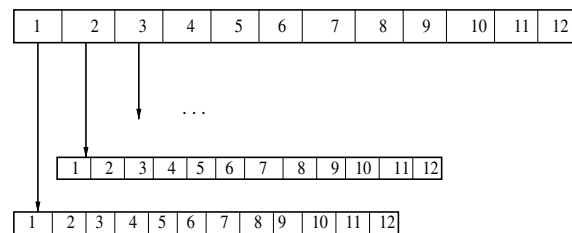


Fig. 4. Example of pTRAD priority queues.

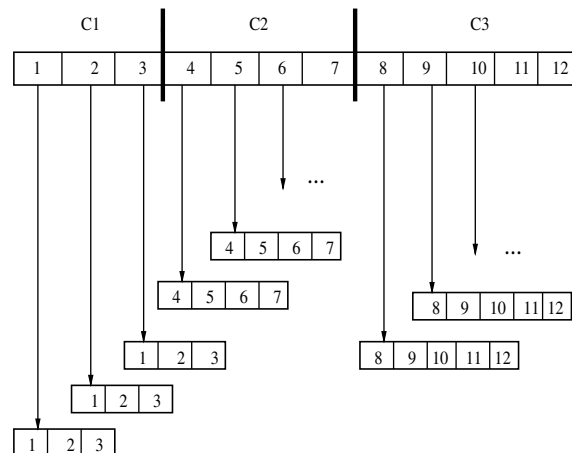


Fig. 5. Example of pPOP priority queues.

Table 1

pTRAD: existing parallel stored matrix algorithm using priority queues

Algorithm: pTRAD Stored Matrix using Priority Queues

Input: Dataset

Output: Dendrogram

Notation: Cclus(P) – chunk of clusters assigned to processor P ;

1. On each processor P
2. Create priority queues for Cclus(P)
3. Repeat while there are clusters to merge
4. On each processor P
5. determine the closest pair (CL_1, CL_2) of Cclus(P)
6. compare (CL_1, CL_2) distance with overall smallest distance
7. update overall closest pair if less (need to avoid race conditions)
8. On designated processor $\bar{P}$
9. merge CL_1 and CL_2
10. On each processor P
11. update the priority queues of Cclus(P)
12. Return

than one processor tries to update the shared overall smallest distance at the same time, a race condition occurs. Therefore a critical region is needed to prevent race conditions. After the overall shared smallest distance is found, the two closest clusters are merged (steps 8–9). The merge involves creating a priority queue for the newly merged cluster. The new priority queue will contain the distances from the newly merged cluster to every other cluster. This step can also be done in parallel by having each processor compute $\frac{N}{p}$ distances. After the merge we also need to update the priority queues of all other clusters. This can be done in parallel (steps 10–11).

3.4. pPOP: proposed parallel stored matrix algorithm using priority queues

Table 2 shows the nested pPOP stored matrix algorithm using priority queues. The initial step of creating the priority queues is done in parallel (Table 2: steps 1–2). Cells are allocated to processors in such a way that each processor computes the priority queues for an average of $\frac{c}{p}$ cells, where c is the total number of cells. To find the overall closest pair of clusters, the cells are allocated between processors using dynamic scheduling. In this way as soon as a processor is done working on its cells, it will go and get another chunk of cells, until all the cells are completed. Each processor will therefore get its own smallest distance, and the cell number where the closest pair of clusters resides (i.e., there will be p smallest distances and p cell numbers that contain the smallest distance) (steps 4–5). From these p distances, the overall smallest distance can be computed by a designated processor (steps 6–7). After the overall smallest distance and the cell that contains the overall closest pair are found, the two closest clusters are merged (step 8). The merge involves creating a priority queue for the newly merged cluster. The new priority queue will contain the distances from the newly merged cluster to every other cluster in the container cell. This step can also be done in parallel by having each processor compute Cclus(C) distances, where Cclus(C) represents a chunk of clusters in the container cell. After the merge we also need to update the priority queues of all other clusters *only* in the container cell. This can be done in parallel by assigning $\frac{\text{Cclus}(C)}{p}$ clusters to each processor. The distance between each cluster and the merged cluster can be computed, and the priority queues updated. If either the agglomerated clusters or the newly merged cluster affect any other cell (i.e., if any of the three are in the δ -region), those cells are also updated. This operation can be done in parallel. Most of the time there is only one affected cell, and very rarely there are two or four affected cells. Therefore it is more efficient to allocate a chunk of the affected cell's clusters to each processor.

In the rest of this section, we compare the CPU time and memory complexities of the sequential and parallel algorithms. For parallel algorithms we choose pPOP and the *existing method* – also referred above as pTRAD. As discussed earlier there are two versions of pPOP – 2-phase and nested. The complexity of nested pPOP is based on 2-phase, and hence we choose 2-phase for the complexity analysis. As in the case of POP, for

Table 2

pPOP: parallel stored matrix algorithm using priority queues

Algorithm: pPOP Stored Matrix Algorithm using Priority Queues (for an iteration of nested algorithm)

Input: Dataset, partitioning scheme (c, δ)

Output: Dendrogram

Notation: Ccell(P) – chunk of cells assigned to processor P ;Cclus(C) – chunk of clusters in cell C ;

1. On each processor P
2. Create priority queues for Ccell(P)
3. Repeat while *overall_min_dist* $< \delta$
4. On each processor P
5. determine the closest pair (CL_1, CL_2) for each cell C in Ccell(P)
6. On designated processor $\bar{P}$
7. find the overall closest pair (CL_1, CL_2) and the cell C that holds it
8. merge CL_1 and CL_2
9. On each processor P
10. update the priority queues of Cclus(C)
11. On designated processor $\bar{P}$
12. determine if any other neighboring cell C is affected by the merging
13. On each processor P
14. for each affected cell C
15. update the priority queues of Cclus(C)
16. Return

the 2-phase pPOP algorithm we assume equal cell sizes, negligible δ region, and negligible phase 2 time. We assume equal load distribution, i.e., each processor is responsible for $\frac{c}{p}$ cells where c is the number of cells and p is number of processors. The chosen sequential algorithms were described earlier in this paper.

3.5. Complexity comparison for dissimilarity matrix type

The pPOP algorithm using dissimilarity matrix works as shown in Table 3. In Step 1 the dissimilarity matrix for each cell is created. As cells are processed in parallel, and assuming no synchronization delays, the total time is the same as the time taken by any one processor. Assuming equal cell size, each cell will have $\frac{N}{c}$ clusters where c is the number of cells. And, assuming equal load distribution each processor is responsible for $\frac{c}{p}$ cells. Thus, the total time for Step 1 is $O\left(\frac{N^2}{cp}\right)$. On the other hand, Step 1 of the existing parallel algorithm requires $O\left(\frac{N^2}{p}\right)$ as the task of computing $O(N^2)$ dissimilarities is naively divided among the p processors. The other critical step in the algorithm is Step 3, in which the smallest distance among all clusters is found. Analyzing as above, the time complexity for this step is also $O\left(\frac{n^2}{cp}\right)$ where n is the remaining number of clusters at that iteration. Compare this to the time taken by the existing parallel algorithm which is $O\left(\frac{n^2}{p}\right)$. Hence the overall time complexity of pPOP is $O\left(\frac{N^3}{cp}\right)$ whereas that for the existing parallel algorithm is $O\left(\frac{N^3}{p}\right)$. The memory complexity of pPOP is $O\left(\frac{N^2}{c}\right)$ as compared to $O(N^2)$ for existing parallel and sequential algorithms. The memory complexities are given within brackets ([]) in Table 3.

It is noteworthy that the reduction factor of $O(c)$ is independent of the number of processors p . That means, pPOP is better than the existing parallel methods because of the independent nature of each cell from all other cells. So, even when several cells are processed by a single processor, pPOP still gains significantly over its counterpart.

3.6. Complexity comparison for priority queue type

In Table 4, Step 1 of pPOP computes priority queues in $O\left(\frac{N^2}{cp}\right)$ time. This is similar to Step 1 of the dissimilarity matrix algorithm (see Table 3). Following the same logic, the time complexity of the existing parallel

Table 3

Dissimilarity matrix algorithm: time [memory] complexities of sequential, existing parallel, and pPOP algorithms

Dissimilarity matrix	Existing sequential	POP	Existing parallel	pPOP	RF
1. Create dissimilarity matrix D	$O(N^2)$ [$O(N^2)$]	$O\left(\frac{N^2}{c}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O\left(\frac{N^2}{p}\right)$ [$O(N^2)$]	$O\left(\frac{N^2}{cp}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O(c)$ [$O(c)$]
2. for $n = 1$ to $N - 1$	$O(N)$	$O(N)$	$O(N)$	$O(N)$	
3. find closest pair	$O(n^2)$	$O\left(\frac{n^2}{c}\right)$	$O\left(\frac{n^2}{p}\right)$	$O\left(\frac{n^2}{cp}\right)$	$O(c)$
4. merge closest pair	$O(1)$	$O(1)$	$O(1)$	$O(1)$	
5. update D	$O(n)$	$O(n)$	$O\left(\frac{n}{p}\right)$	$O\left(\frac{n}{p}\right)$	
Overall complexity	$O(N^3)$ [$O(N^2)$]	$O\left(\frac{N^3}{c}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O\left(\frac{N^3}{p}\right)$ [$O(N^2)$]	$O\left(\frac{N^3}{cp}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O(c)$ [$O(c)$]

RF – reduction factor $\left(= \frac{\text{ExistingParallel}}{\text{pPOP}}\right)$.

algorithm is $O\left(\frac{N^2}{p}\right)$. Another critical step in the algorithm in Table 4 is Step 4 which updates the priority queues of the affected clusters. In pPOP a single priority queue holds $\frac{n}{c}$ elements where n is the remaining number of clusters in that iteration. Hence, due to parallelization the total time complexity of Step 4 is $O\left(\frac{n \log \frac{n}{c}}{p}\right)$. Hence, the overall time complexity of pPOP is $O\left(\frac{N^2 \log \frac{N}{c}}{p}\right)$. Step 4 of the existing parallel algorithm requires $O\left(\frac{n \log n}{p}\right)$ as, unlike pPOP, the size of each priority queue is $O(n)$ for n remaining clusters. Hence, the overall time complexity of the existing parallel algorithm is $O\left(\frac{N^2 \log N}{p}\right)$. The overall reduction factor (RF) is shown to be $O(c) + O\left(\frac{\log N}{\log \frac{N}{c}}\right)$ instead of $O\left(\frac{\log N}{\log \frac{N}{c}}\right)$. The reason is that in the actual implementation the time taken by the Step 1 is substantial. So, an RF of $O(c)$ for Step 1 is quite higher than an RF of $O\left(\frac{\log N}{\log \frac{N}{c}}\right)$ for Step 4. Our experimental results agree with this analysis. Just as for the dissimilarity matrix type, the memory complexity of pPOP for priority queues is also $O\left(\frac{N^2}{c}\right)$ as compared to $O(N^2)$ for existing parallel and sequential algorithms.

4. Experimental results

We performed a number of experiments to study the performance and scalability of our proposed pPOP algorithms. Both stored matrix (priority queues) and stored data (nearest neighbor) types of pPOP were implemented using the nested algorithm. For comparison purposes we chose the parallel algorithms of [23] as existing algorithm. So, although other methods might be able to achieve higher performance than [23] due to the

Table 4

Priority queue algorithm: time [memory] complexities of sequential, existing parallel, and pPOP algorithms

Priority queue	Existing sequential	POP	Existing parallel	pPOP	RF
1. Create priority queues PQ	$O(N^2)$ [$O(N^2)$]	$O\left(\frac{N^2}{c}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O\left(\frac{N^2}{p}\right)$ [$O(N^2)$]	$O\left(\frac{N^2}{cp}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O(c)$ [$O(c)$]
2. for $n = N$ to 2	$O(N)$	$O(N)$	$O(N)$	$O(N)$	
3. find closest pair	$O(n)$	$O(n)$	$O\left(\frac{n}{p}\right)$	$O\left(\frac{n}{p}\right)$	
4. merge & update P	$O(n \log n)$	$O\left(n \log \frac{n}{c}\right)$	$O\left(\frac{n \log n}{p}\right)$	$O\left(\frac{n \log \frac{n}{c}}{p}\right)$	$O\left(\frac{\log n}{\log \frac{n}{c}}\right)$
Overall complexity	$O(N^2 \log N)$ [$O(N^2)$]	$O\left(N^2 \log \frac{N}{c}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O\left(\frac{N^2 \log N}{p}\right)$ [$O(N^2)$]	$O\left(\frac{N^2 \log \frac{N}{c}}{p}\right)$ [$O\left(\frac{N^2}{c}\right)$]	$O(c) + O\left(\frac{\log N}{\log \frac{N}{c}}\right)$ [$O(c)$]

RF – reduction factor $\left(= \frac{\text{ExistingParallel}}{\text{pPOP}}\right)$.

use of advanced architecture and other techniques, they are all similar. This is because for stored matrix type the task of computing and maintaining $O(N^2)$ is common for all methods, and similarly for stored data type the task of computing and maintaining the $O(N)$ nearest neighbors is common. The performance was measured in terms of CPU time, memory space and speedup. We experimented using several real, benchmark, and artificial datasets. All experiments were run on an SGI Origin2000 multiprocessor system which is a shared memory machine consisting of 8 R10000 CPUs running at clock rate of 195 MHz. The secondary cache size is 4MB. We used OpenMP which is an API for directed based parallel programming applications in a shared memory environment [5]. We decided to use it because it is designed for fine-grained parallelism, which was predominant in our algorithm.

4.1. Datasets

Synthetic and benchmark datasets are used for the tests. Synthetic datasets are DS2 (taken from BIRCH [30]), t4.8k, t5.8k, t8.8k, t7.10k (taken from CHAMELEON [18] and CURE [13]). These datasets are shown in Appendix A. The benchmark dataset is SEQUOIA [26] used in DBSCAN [12]. These datasets are chosen because of their different shapes of clusters (e.g., arbitrary and Gaussian) and different noise levels (SEQUOIA has very high noise level).

4.2. Results for synthetic datasets

In Fig. 6 we show the results over the synthetic dataset DS2 (taken from BIRCH [30]) whose size varies from 3K to 60K. The existing stored matrix algorithms require $O(N^2)$ memory, hence we could experiment only with datasets of size up to 5K; on the other hand for pPOP we report results for datasets up to 30K. The number of processors varies from 1 to 8. In Fig. 6(a) and (b) we report the speedups of pPOP. Although the speedup of pPOP is small for smaller datasets, we observe that for larger datasets (30K or higher) the speedup of pPOP improves substantially and approaches linear speedup for dataset of size 60K. Fig. 6(c) and (d) gives the relative speedup of pPOP over the existing algorithm. pPOP is always superior over the existing algorithm because of its efficient partitioning, and independent nature of each cell. The relative speedup increases with data size. Among stored matrix and stored data types, pPOP's performance is much better for stored matrix. It achieves multiple orders of magnitude improvement in computation time over the existing algorithm.

As shown in Figs. 6(c) and (d) the relative speedup, $\frac{\text{ExistingCPU}}{\text{pPOP CPU}}$, decreases as the number of processors increases. This is due to the fact that for a small number of cells, when the number of processors is increased, some processors end up working on cells containing a very small number of clusters, and will therefore spend a lot of time being idle after they are done with the computation in a given iteration. However, as the dataset size increases and/or the number of clusters increases, load balancing among the processors becomes better. This phenomena can be observed in our figures. Although for both 3K and 5K dataset sizes for stored matrix type we observe a relative speedup drop when the number of processors increased from 1 to 8, observe that for 5K the rate of drop in speedup decreased compared with 3K. Although due to the high memory requirements of the existing parallel algorithms we could not test for higher dataset sizes, we postulate that for larger datasets this trend of reduction in relative speedup for more processors will continue to slow down further.

We compared the memory requirements for stored matrix type. For 3K and 5K pPOP reduced the memory requirements by a factor of 97 and 189, i.e., $\frac{\text{Memory_required_by_existing_for_5k_dataset}}{\text{Memory_required_by_pPOP_for_5k_dataset}}$ is 97 and 189 respectively. For stored data type both algorithms require similar amount of memory.

4.3. Results of other synthetic and benchmark datasets

In Fig. 7(a) we show the comparison results for the stored matrix type algorithms over the remaining synthetic and benchmark datasets, i.e., t4.4k, t5.4k, t8.4k, t7.4k and SEQUOIA.4k. Notice that legends in these figures have the following significance. The size of each of t4.4k, t5.4k, t8.4k, t7.4k and SEQUOIA.4k datasets is 4k. In Fig. 7(b) we show the results for t4.8k, t5.8k, t8.8k, t7.10k and SEQUOIA. The size of each of t4.8k, t5.8k and t8.8k datasets is 8k, the size of t7.10k dataset is 10k, and the size of Sequoia dataset is 12k. The reason for using different sizes for each of these datasets for Fig. 7(a) and (b) is that in Fig. 7(a) stored matrix

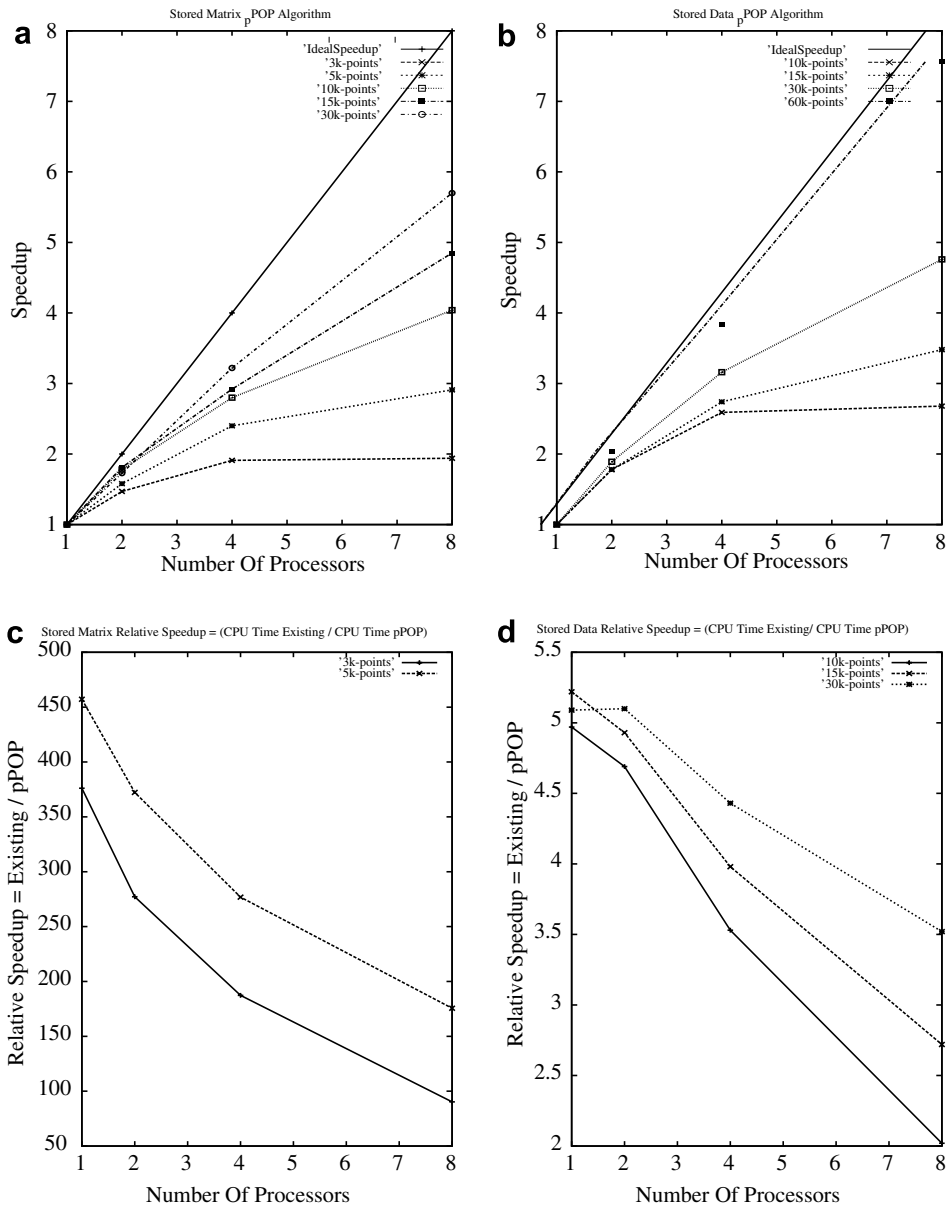


Fig. 6. Results for synthetic datasets DS2.

algorithm allows only limited size data as the memory required for larger data size overflows our system's main memory capacity. But in Fig. 7(b) stored data algorithm is used which can handle much larger data sizes. The results show that pPOP outperforms the existing parallel algorithm by multiple orders of magnitude improvement in computation time for stored matrix algorithm (see Fig. 7(a)). As before, the number of processors varies from 1 to 8. In Fig. 7(b), where we show the relative speedup of pPOP compared to existing algorithm for stored data type, pPOP outperforms the existing algorithm although not as significantly as in the stored matrix type.

5. High-dimensional data and hpc-tree

The above discussion focuses on POP for 2-D data. We proposed a very efficient data structure, called hpc-tree (hyper-plane clustering tree) for high-dimensional data, as a replacement for the axis-parallel partitioning

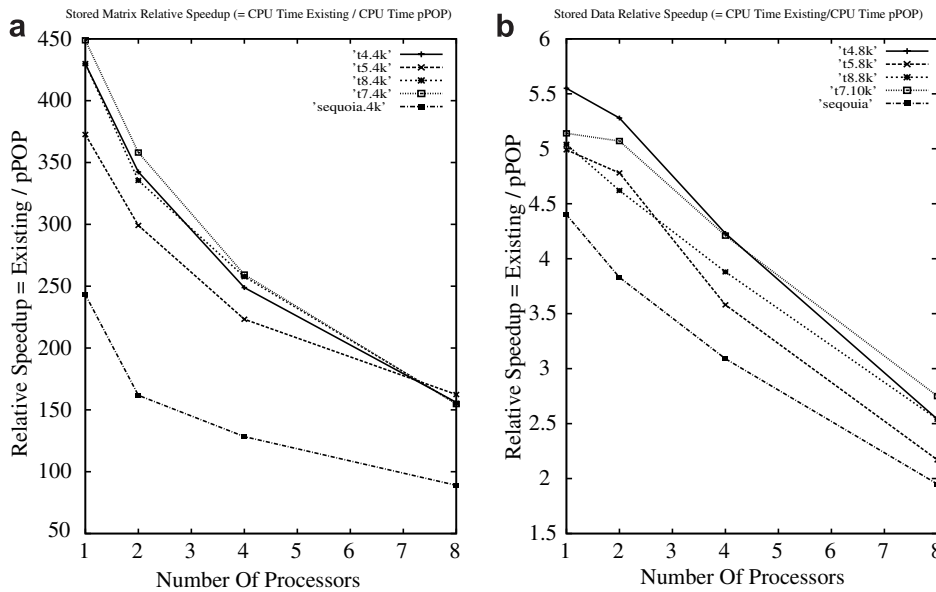


Fig. 7. Results for other synthetic and benchmark datasets.

in [7]. The structures of the two partitioning schemes are very similar, i.e., anything applicable to axis-parallel partitioning for 2-D data is also applicable to hpc-tree partitioning for high-dimensional data. We limit the implementation to 2-D data in this paper and leave the implementation for high-dimensional data as a future extension. In the rest of this section we first give a brief description of the proposed hpc-tree data structure for POP followed by a brief discussion on how to implement hpc-tree for pPOP.

5.1. hpc-tree for POP

High-dimensional data poses problems for the simple axis-parallel partitioning technique discussed before. If a single dimension is used to partition the data, then the δ distance in single dimension will be too small in high-dimensional space which reduces the efficiency drastically. If all (or many) dimensions are used to partition the space and if each is divided into at least two, then for M dimensions there will be 2^M partitions; e.g., for $M = 20$ there will be $2^{20} \approx 10^6$, for $M = 100$ there will be $2^{100} \approx 10^{30}$ (!) partitions. Most of the partitions will be empty, even when the number of data points N is large, leading to extreme waste of memory and CPU time.

Before we discuss how to efficiently partition the high-dimensional data we discuss two important arguments both for and against any such partitioning. These two studies are on nearest neighbor search which is closely linked to the problem of finding clusters in the data. In [28], it is shown that almost all partitioning schemes for efficient nearest neighbor search in high-dimensional data fail if the data is distributed uniformly. A clustering application implicitly assumes that data is not uniformly distributed but has clusters in it, and so the assumption of uniform distribution will be irrelevant for this paper. The other study in [2] argues that some partitioning techniques can still be efficient if there are clusters in high-dimensional data. The authors argue that as dimensionality increases the points within a cluster must be increasingly closer than points in different clusters. This can be justified as follows. The authors show in the paper that the distance between any pair of points approaches the maximum data distance if data points are uniformly distributed. So, the gap between intra-cluster and inter-cluster distances increases with increasing dimensionality. With respect to the topic of this paper it can be safely concluded that efficient partitioning techniques are possible for high-dimensional clustering.

Our goal is to construct a partitioning scheme that handles high-dimensionality well. Some additional desirable properties of a partitioning scheme that incorporates POP are: (1) $|\delta|$ should be small, (2) the time spent in distributing points and determining affected partitions should be small, and (3) partitions should not hold too

many or too few points. In OptiGrid [15] an optimal hyper-plane partitioning scheme is defined for implementing grid-based clustering. This is not applicable for POP as a main goal of OptiGrid is not to break any cluster. This objective can distribute a large number of points in a single cell violating criterion 3 and resulting in poorer POP performance. An efficient way to satisfy criterion 2 is a tree structure that needs $O(\log n')$ number of comparisons on average to distribute a point and to determine the affected partitions where n' is the number of nodes in the tree. One way to satisfy criterion 1 is to use a Voronoi diagram which partitions the space into Voronoi cells, directed by a set of *split* points $Q = q_1, q_2, \dots$ such that for each cell corresponding to split point q_i , the points x in that cell are nearer to q_i than to any other split point in Q . (See [25] for details on Voronoi diagram.) Hence, by constructing a hierarchy of Voronoi diagrams, all the above criteria can be satisfied. We call such a structure as *hpc* – *hyper plane clustering* – *tree* where the top or root node gives a brief summary of the whole data and is split to many Voronoi cells which are split as well and so on.

A challenging aspect of the *hpc*-tree is the selection of split points. In [4] a hierarchical structure of a Voronoi diagram called GNAT is proposed for efficient nearest neighbor search in high-dimensional data. The authors have argued that for efficient neighborhood search in high-dimensional data, a hierarchical geometrical structure of data is more suitable than structures that do not consider the intrinsic geometry of the data. In GNAT, split points are chosen randomly and the user is required to specify the number of split points in each level of the hierarchy. Wrong selection of split points results in an unbalanced tree resulting in disproportionate distribution among the cells and poor POP performance. An ideal split point should be a summary of the points that belong to the Voronoi cells that are children of the Voronoi cell corresponding to the split point. In the literature, there are different methods of creating a summary of data (e.g., CF-tree (BIRCH [30]), and Bradley et al's method [3]). CF-tree is more suitable for *hpc*-tree because of its hierarchical structure. An artifact of CF-tree is that it does not guarantee that a parent node is the closest parent of all of its children which is necessary for *hpc*-tree. We address this problem by removing the split points in each level of the hierarchy that have distance less than δ from some other split point.

The leaf nodes (a node consists of a split point among other information) of *hpc*-tree are the cells of POP. Starting with the root, for each data point, the path that contains the nearest child is descended until the leaf level is reached. If a point does not belong to δ -boundary of any node of the path, then it is assigned to only one leaf node as a non-boundary point. Otherwise, each affected node is descended and the point is assigned to all the affected leaf nodes as a boundary point, and so on. A simple example of *hpc*-tree is illustrated in Fig. 8. In the 1st level only the root exists which is not shown in the figure. In level 2, there are 9 nodes with heavier split points and thicker boundary lines. The 3rd level has finer split points and thinner boundary lines. The dashed lines show the boundary of width δ for one branch of the *hpc*-tree. Unlike the axis-parallel partitioning where it is easy to determine the neighboring cells of a data point, in *hpc*-tree each boundary point stores pointers to the

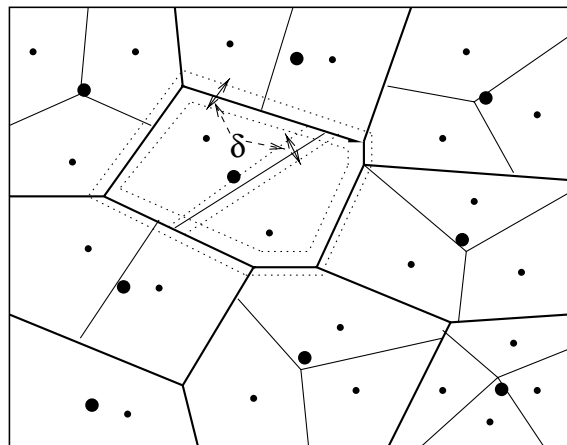


Fig. 8. A simple *hpc*-tree.

affected partitions. Nested partitioning can use the same hpc-tree for different δ if the minimum separation between the split points is not less than the new δ ; otherwise, the tree is reconstructed with the new δ .

5.2. hpc-tree for pPOP

There are two main tasks in parallelizing hpc-tree: (a) assigning the data points to their respective Voronoi cells in the leaf level of the hpc-tree, and (b) partitioning these cells among the processors. Task (a) can be divided among the processors equally, i.e., each processor can handle equal number of data points. For each data point, the assigned processor will independently determine the leaf level Voronoi cell using the hpc-tree structure. Task (b) is performed just the same way as axis-parallel partitioning. A cell is assigned to one processor which handles all merges in the cell, and all cells are divided among the processors.

6. Conclusion and future directions

In this paper we proposed pPOP, an efficient parallel hierarchical clustering algorithm. Our theoretical analysis and experiments showed that, for both stored matrix and stored data types, pPOP outperforms the existing algorithms significantly both in CPU time and memory requirements. Particularly for the stored matrix algorithm the gain has been of multiple orders of magnitude. This is achieved by exploiting a 90–10 rule of HAC which states that in a dendrogram most levels from the bottom merge pairs of very small clusters separated by a very small portion of the maximum closest pair distance. The data space was partitioned by partially overlapping cells, and each cell could be processed to a large extent independent of other such cells without affecting accuracy. This factor allowed to distribute and process the cells independent of other cells.

There are several directions of future work. An obvious extension of this work is to implement the hpc-tree [7] for pPOP for high-dimensional datasets. Another direction is to implement our pPOP algorithms in an MPI architecture and to compare the results with those obtained for shared memory. Other directions include applying this efficient algorithm to practical datasets such as gene expression data.

Appendix A. Datasets

A number of 2-D datasets having clusters of arbitrary shape and multiple resolution are taken from CHAMELEON [18], CURE [13], and BIRCH [30] (see Fig. A.1).

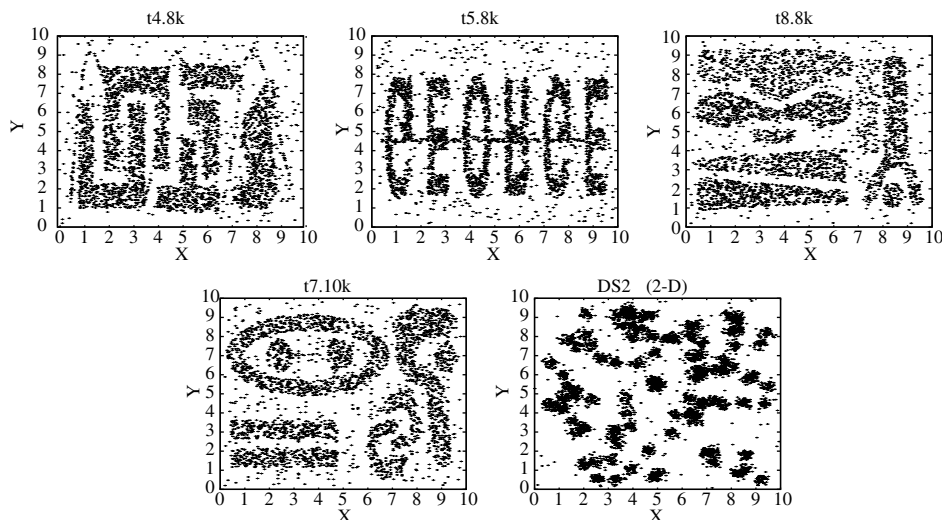


Fig. A.1. Clusters with arbitrary shape and multiple resolution [18,30].

References

- [1] M.R. Anderberg, Cluster Analysis for Applications, Academic Press, New York, 1973.
- [2] K. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft, When is “nearest neighbor” meaningful? in: Proceedings of the International Conference on Database Theory (ICDT), 1999, pp. 217–235.
- [3] P.S. Bradley, U. Fayyad, C. Reina, Scaling clustering algorithms to large databases, in: Proceedings of the 4th International Conference on Knowledge Discovery and Data Mining (KDD), 1998, pp. 9–15.
- [4] S. Brin, Near neighbor search in large metric spaces, in: Proceedings of the International Conference on Very Large Databases (VLDB), 1995.
- [5] R. Chandra, L. Dagum, D. Kohr, D. Maydan, J. McDonald, R. Menon (Eds.), Parallel Programming in OpenMP, Morgan Kaufman Publishers, 2000.
- [6] M. Dash, Efficient clustering for knowledge discovery in large high-dimensional databases, Ph.D. thesis, Department of Computer Science, School of Computing, National University of Singapore, 2002. Available from: <http://www.ece.northwestern.edu/~manoranj/thesis_phd.ps>.
- [7] M. Dash, H. Liu, P. Scheuermann, K.L. Tan, Fast hierarchical clustering and its validation, Data and Knowledge Engineering 44 (1) (2003) 109–138.
- [8] M. Dash, S. Petrutiu, P. Scheuermann, Efficient parallel hierarchical clustering, in: Proceedings of 10th International Conference on Parallel and Distributed Computing (Euro-Par), Springer-Verlag, 2004, pp. 363–371.
- [9] M. Dash, K.L. Tan, H. Liu, Efficient yet accurate clustering, in: Proceedings of IEEE International Conference on Data Mining (ICDM), 2001, pp. 99–106.
- [10] W.H.E. Day, H. Edelsbrunner, Efficient algorithms for agglomerative hierarchical clustering methods, Journal of Classification 1 (1) (1984) 7–24.
- [11] I.S. Dhillon, D.M. Modha, Large-scale parallel data mining, Lecture Notes in Artificial Intelligence 1759 (2000) 245–260.
- [12] M. Ester, H.P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: Proceedings of International Conference on Knowledge Discovery and Data Mining (KDD), 1996, pp. 226–231.
- [13] S. Guha, R. Rastogi, K. Shim, CURE: an efficient clustering algorithm for large databases, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 1998, pp. 73–84.
- [14] Jiawei Han, Micheline Kamber, Data Mining: Concepts and Techniques, Morgan Kaufmann Publishers, 2000.
- [15] A. Hinneburg, D.A. Keim, Optimal grid-clustering: towards breaking the curse of dimensionality clustering, in: Proceedings of International Conference on Very Large Databases (VLDB), 1999.
- [16] A.K. Jain, R.C. Dubes, Clustering methods and algorithms, in: Algorithm for Clustering Data, Prentice-Hall Advanced Reference Series, 1988.
- [17] A.K. Jain, M.N. Murty, P.J. Flynn, Data clustering: a review, ACM Computing Surveys 31 (3) (1999) 264–323.
- [18] G. Karypis, E-H. Han, V. Kumar, CHAMELEON: a hierarchical clustering algorithm using dynamic modeling, IEEE Computer 32 (1999) 68–75.
- [19] X. Li, Parallel algorithms for hierarchical clustering and cluster validity, IEEE Transactions on Pattern Analysis and Machine Intelligence 12 (1990) 1088–1092.
- [20] X. Li, Z. Fang, Parallel clustering algorithms, Parallel Computing 11 (1989) 275–290.
- [21] G. Milligan, M. Cooper, An examination of procedures for determining the number of clusters in a data set, Psychometrika 50 (1985) 159–179.
- [22] H. Nagesh, S. Goil, A. Choudhary, PMAFIA: a scalable parallel subspace clustering algorithm for massive datasets, in: Proceedings of the International Conference on Parallel Processing, 2000, pp. 21–24.
- [23] C.F. Olson, Parallel algorithms for hierarchical clustering, Parallel Computing 21 (1995) 1313–1325.
- [24] E. Omiecinski, P. Scheuermann, A parallel algorithm for record clustering, ACM Transactions on Database Systems 15 (4) (1990) 599–624.
- [25] F.P. Preparata, M.I. Shamos (Eds.), Computational Geometry: An Introduction, Springer-Verlag, 1985.
- [26] M. Stonebraker, J. Frew, K. Gardels, J. Meredith, The SEQUOIA 2000 storage benchmark, in: Proceedings of ACM SIGMOD International Conference on Management of Data, 1993, pp. 2–11.
- [27] J.H. Ward Jr., Hierarchical grouping to optimize an objective function, Journal of the American Statistical Association 58 (1963) 236–244.
- [28] R. Weber, H.-J. Schek, S. Blott, A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces, in: Proceedings of the International Conference on Very Large Databases (VLDB), 1998, pp. 194–205.
- [29] C.H. Wu, S.J. Horng, H.R. Tsai, Efficient parallel algorithms for hierarchical clustering on arrays with reconfigurable optical buses, Journal of Parallel and Distributed Computing 60 (2000) 1137–1153.
- [30] T. Zhang, R. Ramakrishnan, M. Livny, BIRCH: an efficient data clustering method for very large databases, in: Proceedings of ACM SIGMOD Conference on Management of Data, Montreal, Canada, June 1996, pp. 103–114.